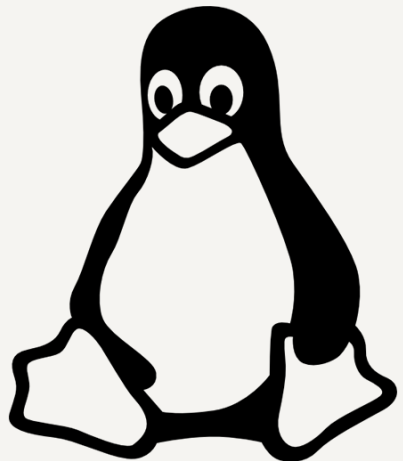


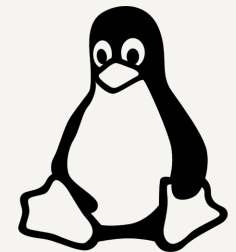
Procesos, Hilos y Sincronización

JUAN PABLO CARVAJAL GIRALDO - 525717

WILSON ANDRES MORENO VELANDIA - 455121



Sincronización En Sistemas distribuidos



Acordar el ORDEN

San Francisco
\$1 111

Nueva York
\$1 110

mismas dos operaciones
llegadas en distinto orden

**Ninguna réplica se equivocó.
El sistema está roto igual.**

No hay un orden correcto:
hay que fabricar un acuerdo

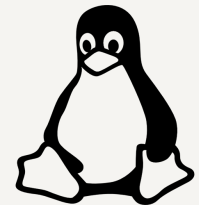
Coordinar el ACCESO

proceso A

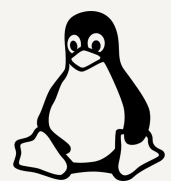
proceso B

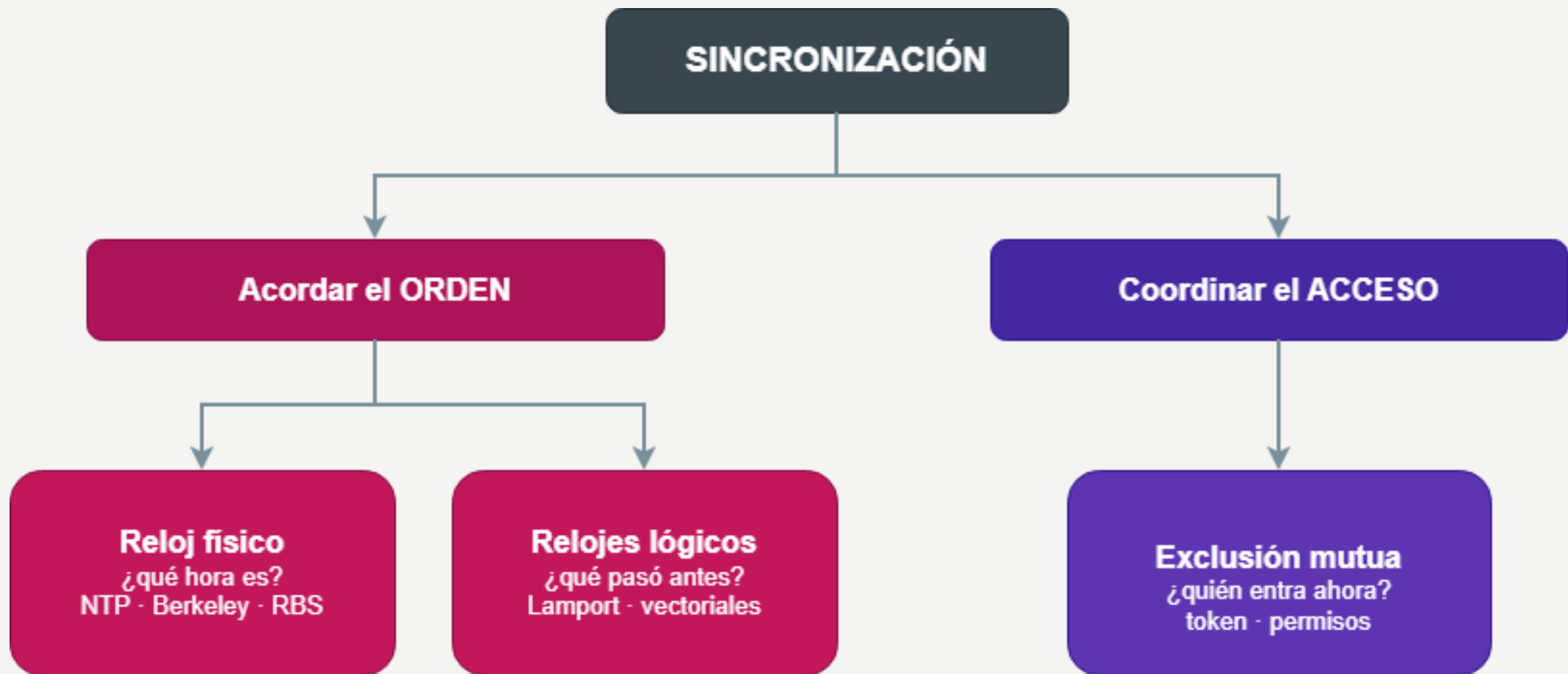
recurso compartido

Sin memoria común no hay
candados ni semáforos:
solo mensajes

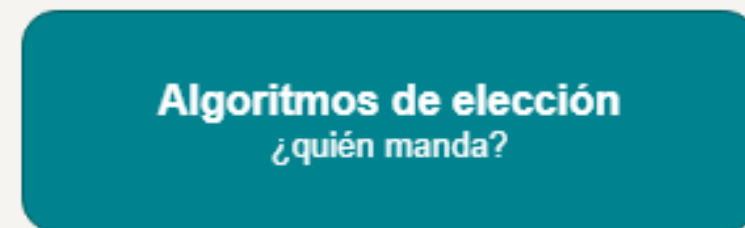
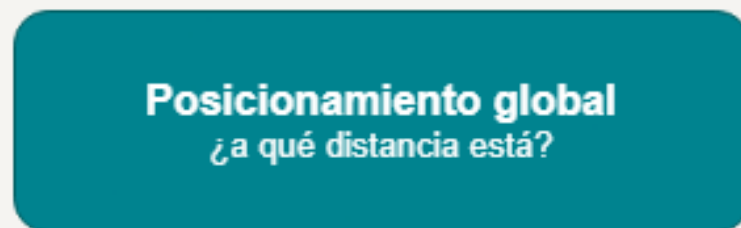


Tipologías de sincronización existentes





habilitan a las anteriores



Exclusión mutua §6.3

Basadas en TOKEN

Anillo de token

1 a ∞ mensajes
si se pierde, difícil detectarlo

Basadas en PERMISOS

Centralizado

3 mensajes
1 punto de falla

Descentralizado

3mk mensajes
iniciación

Distribuido

2(n-1) msgs
n puntos de falla



Técnicas de sincronización



Técnica	Garantiza	Coste	Problema	Caso de uso
6.1 NTP / Cristian	Hora aprox. UTC (1–50 ms)	2 msgs por sondeo ($O(1)$)	Red asimétrica	No, precisión insuficiente
6.2 Lamport	Si a ocurre antes que b, $C(a) < C(b)$	$O(1)$ entero por mensaje	No capta causalidad	Sí, numera cada mensaje
6.2 Vectorial	Causalidad exacta	$O(n)$ enteros por mensaje	Membresía variable	No, da un orden más débil
6.3 Centralizado	Uno a la vez; justo por orden de llegada	3 msgs ($O(1)$)	1 punto de falla	Sí, reparte los turnos
6.3 Distribuido	Uno a la vez; determinista	$2(n-1)$ msgs	n puntos de falla	No, 12 msgs con 7 usuarios
6.3 Anillo de token	Uno a la vez; sin inanición	1 a sin límite	Token perdido	No, hay que esperar el turno
6.5 Abusón (elección)	Acuerdo sobre el coordinador	$O(n^2)$	Fallo en la elección	No aplica



Implementación

Caso de uso



HILO

SEMÁFORO

1 el único permiso: solo un hilo puede tenerlo

SERVIDOR · un proceso Java

Cliente A
proceso Go

envía "hola"

Cliente B
proceso Python

envía "buenas"

HILOS DEL SERVIDOR · 1 principal + 2 por cada cliente conectado

hilo PRINCIPAL
accept() en bucle
solo acepta

hilo LECTOR A
read() del socket
solo recibe

hilo ESCRITOR A
take() del buzón
solo envía

hilo LECTOR B
read() del socket
solo recibe

hilo ESCRITOR B
take() del buzón
solo envía

un hilo, una dirección: si uno leyera y escribiera, no podría entregar mientras espera

LOS LECTORES COMPITEN POR EL RECURSO COMPARTIDO · el contador seq

COLA
peticiones aplazadas

SEMÁFORO
Semaphore(1, fair)

REGIÓN CRÍTICA
numerar + depositar

TURNO 1

hilo LECTOR B
dormido, sin gastar CPU

permisos: 0
acquire() de B
no hay -> a la cola

hilo LECTOR A
seq: 40 -> 41
deposita en los 2 buzones

1

A llama a release() · devuelve el permiso

TURNO 2

vacía

permisos: 1 -> 0
despierta al
PRIMERO de la cola

hilo LECTOR B
seq: 41 -> 42
deposita en los 2 buzones

1

recibe
[41] hola
[42] buenas

buzón A (cola bloqueante)
hilo ESCRITOR A
vacía y envía · SIN semáforo

buzón B (cola bloqueante)
hilo ESCRITOR B
vacía y envía · SIN semáforo

recibe
[41] hola
[42] buenas



Sincronización en Git



**Repositorio local
(tu máquina)**



**Repositorio remoto
(GitHub / historial compartido)**



push

pull / fetch

El orden lo da la cadena de padres, no un reloj físico (relojes lógicos de Lamport, cap. 6.2)

Referencias

- [1] Serrano Gil, L. J. (2026). *Mediación docente: Sistemas Distribuidos 2026-20*. Universidad Pontificia Bolivariana, Seccional Bucaramanga.
- [2] Tanenbaum, A. S., y van Steen, M. (2008). *Sistemas distribuidos: principios y paradigmas* (2ª ed.). Pearson Educación.
- [3] Object Management Group (2017). *OMG Unified Modeling Language (UML), Version 2.5.1*. Documento formal/2017-12-05.



GRACIAS

